Andrew J. Anagnost College of Engineering & Computer Science

CSUN. CALIFORNIA STATE UNIVERSITY NORTHRIDGE

COMP 542 – Machine Learning

Professor: Dr. Verma

Technical Documentation of Application

Adult Census Income Classification System

Prepared by:

Daniel Graham

Jean Paul Collazo

Janeth Lucero Garcia Rodriguez

August 18th, 2026

Los Angeles, California

Table of Contents

1.0 Introduction 3
1.1 Purpose 3
1.2 Definitions, Acronyms, and Abbreviations 5
2.0 Technical Documentation 7
2.1 Important Files 8
2.2 Source Code Documentation 10
2.3 Application Workflow 13
Conclusion 17
References 18

1.0 Introduction

This document provides the technical documentation for the Adult Census Income Classification System, a supervised machine learning application developed as the final group project for COMP 542 – Machine Learning at California State University, Northridge.

The application uses the Adult Census Income dataset from the UCI Machine Learning Repository to predict whether an individual's annual income is greater than $50,000 based on demographic and employment-related attributes. The project follows a complete machine learning workflow that includes data exploration, preprocessing, feature engineering, model development, hyperparameter tuning, performance evaluation, and comparison of multiple classification algorithms.

This technical documentation describes the architecture of the application, the major project files, the software components developed by the team, and the purpose of each module. The document is intended to support future maintenance, testing, enhancement, and understanding of the application by providing a clear overview of its implementation and organization.

1.1 Purpose

The purpose of this document is to provide the technical documentation for the **Adult Census Income Classification System** developed as the final group project for **COMP 542 – Machine Learning**. This documentation serves as a technical reference for understanding the application's architecture, implementation, source code organization, preprocessing workflow, machine learning models, and evaluation methodology. It is intended to support future maintenance, testing, enhancement, and continued development of the application.

The project was completed collaboratively by **Jean Paul Collazo**, **Janeth Lucero Garcia Rodriguez**, and **Daniel Graham**. Throughout the project, the team jointly defined the machine learning problem, selected the dataset, established the preprocessing strategy, designed the evaluation methodology, compared model performance, and interpreted the experimental results. Development followed a common workflow to ensure that all machine learning models were trained and evaluated using consistent preprocessing techniques, performance metrics, and validation procedures.

To efficiently divide the implementation work, each team member assumed primary responsibility for specific machine learning models while contributing to the overall integration of the project. Daniel Graham implemented and evaluated the **K-Nearest Neighbors (KNN)** and **XGBoost** classifiers. Janeth Lucero Garcia Rodriguez performed the exploratory data analysis and implemented the **Logistic Regression** and **Naive Bayes** classifiers. Jean Paul Collazo implemented and evaluated the **Random Forest** and **Neural Network (Multi-Layer Perceptron)** classifiers. Together, the team compared the performance of all six models using common evaluation metrics, including accuracy, precision, F1-score, ROC-AUC, and cross-validation accuracy, both before and after hyperparameter tuning.

This document summarizes the technical components of the completed application, describes the major project files and software modules, and documents the responsibilities associated with each implementation component. Its objective is to provide sufficient technical information for future developers, instructors, or project team members to understand, maintain, and extend the application.

1.2 Definitions, Acronyms, and Abbreviations

The following definitions, acronyms, and abbreviations are used throughout this technical documentation.

Table 1. Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
ML	Machine Learning, a branch of artificial intelligence that enables computers to learn patterns from data and make predictions without being explicitly programmed.
UCI	University of California, Irvine. The institution that maintains the UCI Machine Learning Repository, which provides the Adult Census Income dataset used in this project.
Adult Census Income Dataset	A public dataset containing demographic and employment-related information used to predict whether an individual's annual income exceeds $50,000.
Feature	An input variable used by a machine learning model to make predictions (e.g., age, education, occupation, hours per week).
Target Variable	The output variable that the model attempts to predict. In this project, the target variable is the individual's income category (<=50K or >50K).
Supervised Learning	A machine learning approach where models are trained using labeled data containing both input features and known target values.
Binary Classification	A classification problem with only two possible output classes. This project predicts whether annual income is <=50K or >50K.
EDA	Exploratory Data Analysis. The process of examining and visualizing data before model development to understand its characteristics and identify potential issues.
Pipeline	A sequence of preprocessing and machine learning operations executed in a defined order using Scikit-Learn.

ColumnTransformer	A Scikit-Learn utility that applies different preprocessing techniques to numerical and categorical features.
One-Hot Encoding	A preprocessing technique that converts categorical variables into binary numerical features suitable for machine learning algorithms.
StandardScaler	A preprocessing method that standardizes numerical features by removing the mean and scaling them to unit variance.
Train-Test Split	The process of dividing the dataset into training and testing subsets for model development and evaluation.
Cross-Validation (CV)	A validation technique that repeatedly partitions the training data to estimate how well a model generalizes to unseen data.
GridSearchCV	A Scikit-Learn method used to identify the best combination of hyperparameters through exhaustive search and cross-validation.
Hyperparameter	A model configuration value specified before training that controls the learning process (e.g., number of trees, learning rate, number of neighbors).
KNN	K-Nearest Neighbors, a supervised classification algorithm that predicts a class based on the nearest training observations.
XGBoost	Extreme Gradient Boosting, an ensemble learning algorithm based on sequentially constructed decision trees.
Random Forest	An ensemble machine learning algorithm that combines multiple decision trees to improve predictive performance and reduce overfitting.
MLP	Multi-Layer Perceptron, a feed-forward artificial neural network used for classification tasks.
Logistic Regression	A supervised classification algorithm that estimates the probability that an observation belongs to a particular class.
Naive Bayes	A probabilistic classification algorithm based on Bayes' Theorem and the assumption of conditional independence among features.
Accuracy	The proportion of correctly classified observations among all predictions.
Precision	The proportion of predicted positive observations that are actually positive.
Recall	The proportion of actual positive observations correctly identified by the model.

F1-Score	The harmonic mean of precision and recall, commonly used for evaluating classification models.
ROC-AUC	Receiver Operating Characteristic – Area Under the Curve. A performance metric that measures a classifier's ability to distinguish between classes across different classification thresholds.
Confusion Matrix	A table that summarizes a classification model's performance by showing true positives, true negatives, false positives, and false negatives.

Table 1. Definitions, Acronyms, and Abbreviations

2.0 Technical Documentation

The Adult Census Income Classification System was developed using Python and Jupyter Notebooks to implement, evaluate, and compare multiple supervised machine learning classification models. The application follows a structured end-to-end machine learning workflow that includes data loading, exploratory data analysis, data preprocessing, feature transformation, model training, hyperparameter tuning, performance evaluation, and comparison of classification results.

To maintain consistency across the project, all machine learning models were developed using a common preprocessing strategy and evaluated using the same performance metrics. Numerical and categorical features were processed through Scikit-Learn preprocessing pipelines before being supplied to the classification algorithms. Each model was then evaluated using accuracy, precision, F1-score, ROC-AUC, confusion matrices, and cross-validation accuracy to provide a consistent basis for performance comparison.

The following sections describe the major project files, software modules, preprocessing components, machine learning models, and supporting libraries that collectively form the completed application.

2.1 Important Files

The Adult Census Income Classification System consists of several Jupyter notebooks, supporting documentation, and external resources that collectively implement the machine learning workflow. Each file serves a specific purpose within the application and contributes to the overall development, evaluation, and comparison of the classification models.

Table 2. Important Files of the Adult Census Income Classification System

File	Primary Programmer(s)	Description
Adult Census Income Dataset	Team	The primary dataset obtained from the UCI Machine Learning Repository. It contains demographic and employment-related attributes used to predict whether an individual's annual income exceeds $50,000.
janeth_garcia.ipynb	Janeth Lucero Garcia Rodriguez	Performs exploratory data analysis (EDA), data preprocessing, feature preparation, and implements the Logistic Regression and Naive Bayes classification models. The notebook includes model evaluation, cross-validation, and hyperparameter tuning using GridSearchCV.
Jean Paul_draft_updated.ipynb	Jean Paul Collazo	Implements the Random Forest and Neural Network (Multi-Layer Perceptron) classifiers using the shared preprocessing pipeline. The notebook performs model training, prediction, performance evaluation, cross-validation, confusion matrix generation, and hyperparameter optimization.

Daniel.ipynb	Daniel Graham	Implements the K-Nearest Neighbors (KNN) and XGBoost classification models. The notebook includes feature preprocessing, model training, performance evaluation, cross-validation, and hyperparameter tuning for both algorithms.
README.md	Team	Provides an overview of the project, identifies the project members, documents the dataset source, summarizes the machine learning models implemented, and presents the consolidated performance results before and after hyperparameter tuning.
Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow	Reference Material	Course reference used to guide the overall machine learning workflow, including data preprocessing, supervised learning, model evaluation, cross-validation, and hyperparameter tuning concepts.

Table 2. Important Files of the Adult Census Income Classification System

In addition to the project files, the application relies on several third-party Python libraries that support data processing, visualization, model development, and evaluation. The primary libraries include:

- **Pandas** – Data loading, cleaning, and manipulation.
- **NumPy** – Numerical computing and array operations.
- **Matplotlib** – Data visualization and plotting.
- **Seaborn** – Statistical data visualization.

- **Scikit-Learn** – Machine learning algorithms, preprocessing pipelines, model evaluation, cross-validation, and hyperparameter tuning.
- **XGBoost** – Gradient boosting classification algorithm used for the XGBoost model.

2.2 Source Code Documentation

Module 1 – Dataset Loading and Data Preparation

Programmer(s): Jean Paul Collazo, Janeth Lucero Garcia Rodriguez, and Daniel Graham

Brief Description:

This module loads the Adult Census Income dataset into a Pandas DataFrame and prepares the data for machine learning. It defines the dataset schema, separates predictor variables from the target variable, converts the income labels into binary classes, and prepares the dataset for preprocessing and model development.

Module 2 – Exploratory Data Analysis (EDA)

Primary Programmer: Janeth Lucero Garcia Rodriguez

Brief Description:

This module performs exploratory analysis of the dataset before model development. The analysis includes inspection of data dimensions, missing values, duplicate records, feature distributions, class balance, descriptive statistics, and visualizations that help identify patterns and potential data quality issues. The results guided the preprocessing decisions used throughout the project.

Module 3 – Data Preprocessing Pipeline

Programmer(s): Jean Paul Collazo, Janeth Lucero Garcia Rodriguez, and Daniel Graham

Brief Description:

This module prepares the dataset for machine learning by applying separate preprocessing operations to numerical and categorical features. Numerical attributes are standardized using **StandardScaler**, while categorical attributes are transformed using **One-Hot Encoding**. The preprocessing workflow is implemented using Scikit-Learn's **Pipeline** and **ColumnTransformer**, ensuring that every classification model receives the same transformed input data.

Module 4 – K-Nearest Neighbors (KNN) Classifier

Primary Programmer: Daniel Graham

Brief Description:

This module implements the K-Nearest Neighbors classification algorithm for predicting annual income categories. The implementation includes model training, prediction, performance evaluation, five-fold cross-validation, and hyperparameter optimization using GridSearchCV to identify the most effective number of neighbors and distance metric.

Module 5 – XGBoost Classifier

Primary Programmer: Daniel Graham

Brief Description:

This module implements the XGBoost classification model using gradient boosting decision trees. The notebook trains the model, evaluates predictive performance, performs cross-validation, and applies hyperparameter tuning to optimize parameters such as the number of estimators, learning rate, tree depth, and subsampling strategy.

Module 6 – Random Forest Classifier

Primary Programmer: Jean Paul Collazo

Brief Description:

This module implements the Random Forest ensemble classifier using the shared preprocessing pipeline. The module trains the model, generates predictions, evaluates performance using multiple classification metrics, performs five-fold cross-validation, and applies GridSearchCV to optimize the Random Forest hyperparameters.

Module 7 – Neural Network (Multi-Layer Perceptron)

Primary Programmer: Jean Paul Collazo

Brief Description:

This module implements a Multi-Layer Perceptron (MLP) Neural Network classifier for binary income prediction. The implementation includes model training, prediction, confusion matrix generation, classification reports, cross-validation, and hyperparameter tuning to improve classification performance.

Module 8 – Logistic Regression Classifier

Primary Programmer: Janeth Lucero Garcia Rodriguez

Brief Description:

This module implements the Logistic Regression classifier using the shared preprocessing pipeline. The classifier estimates the probability that an individual belongs to the income category greater than $50,000 and is evaluated using multiple classification metrics before and after hyperparameter tuning.

Module 9 – Naive Bayes Classifier

Primary Programmer: Janeth Lucero Garcia Rodriguez

Brief Description:

This module implements the Gaussian Naive Bayes classifier for binary income prediction. The implementation includes preprocessing, model training, prediction, performance evaluation, cross-validation, and optimization of the variance smoothing parameter using GridSearchCV.

Module 10 – Model Evaluation and Performance Comparison

Programmer(s): Jean Paul Collazo, Janeth Lucero Garcia Rodriguez, and Daniel Graham

Brief Description:

This module provides a unified evaluation framework for comparing all six machine learning models. Each classifier is evaluated using the same performance metrics, including **Accuracy**, **Precision**, **Recall**, **F1-Score**, **ROC-AUC**, **Confusion Matrix**, and **Cross-Validation Accuracy**. Baseline and hyperparameter-tuned models are compared to determine the most effective classifier for the Adult Census Income prediction task.

2.3 Application Workflow

The Adult Census Income Classification System follows a structured end-to-end machine learning workflow that ensures all classification models are developed and evaluated using a consistent methodology. The workflow begins with loading the Adult Census Income dataset, followed by exploratory data analysis and preprocessing to prepare the data for machine learning. Numerical and categorical features are transformed through a shared preprocessing pipeline before being used for model training.

Once preprocessing is complete, the dataset is divided into training and testing subsets using a stratified train-test split to preserve the distribution of the target classes. Six supervised machine learning algorithms are then trained using the processed data: K-Nearest Neighbors (KNN), XGBoost, Random Forest, Neural Network (Multi-Layer Perceptron), Logistic Regression, and Naive Bayes.

Each model is evaluated using a common set of performance metrics, including Accuracy, Precision, Recall, F1-Score, ROC-AUC, Confusion Matrix, and Cross-Validation Accuracy. After establishing baseline performance, GridSearchCV is applied to optimize the hyperparameters of each classifier. Finally, the tuned models are compared to identify the algorithm that provides the best predictive performance for the Adult Census Income classification problem.

Interactive Prediction Feature

As a final component of the application, an interactive prediction cell was implemented within the Jupyter Notebook environment. This feature allows users to manually enter demographic and employment-related information, including variables such as age, education, occupation, hours worked per week, capital gain, and other selected features. The entered values are automatically processed through the same preprocessing pipeline used during model training before being supplied to the selected machine learning classifier.

The application then predicts whether the individual's estimated annual income falls within the <=50K or >50K income category. This functionality demonstrates the practical application of the trained machine learning models by allowing users to perform real-time inference using custom input values. The prediction cell also verifies that the preprocessing pipeline and trained

classification model can be successfully applied to previously unseen data, simulating how the application would operate in a real-world deployment.

The overall application workflow is illustrated below.

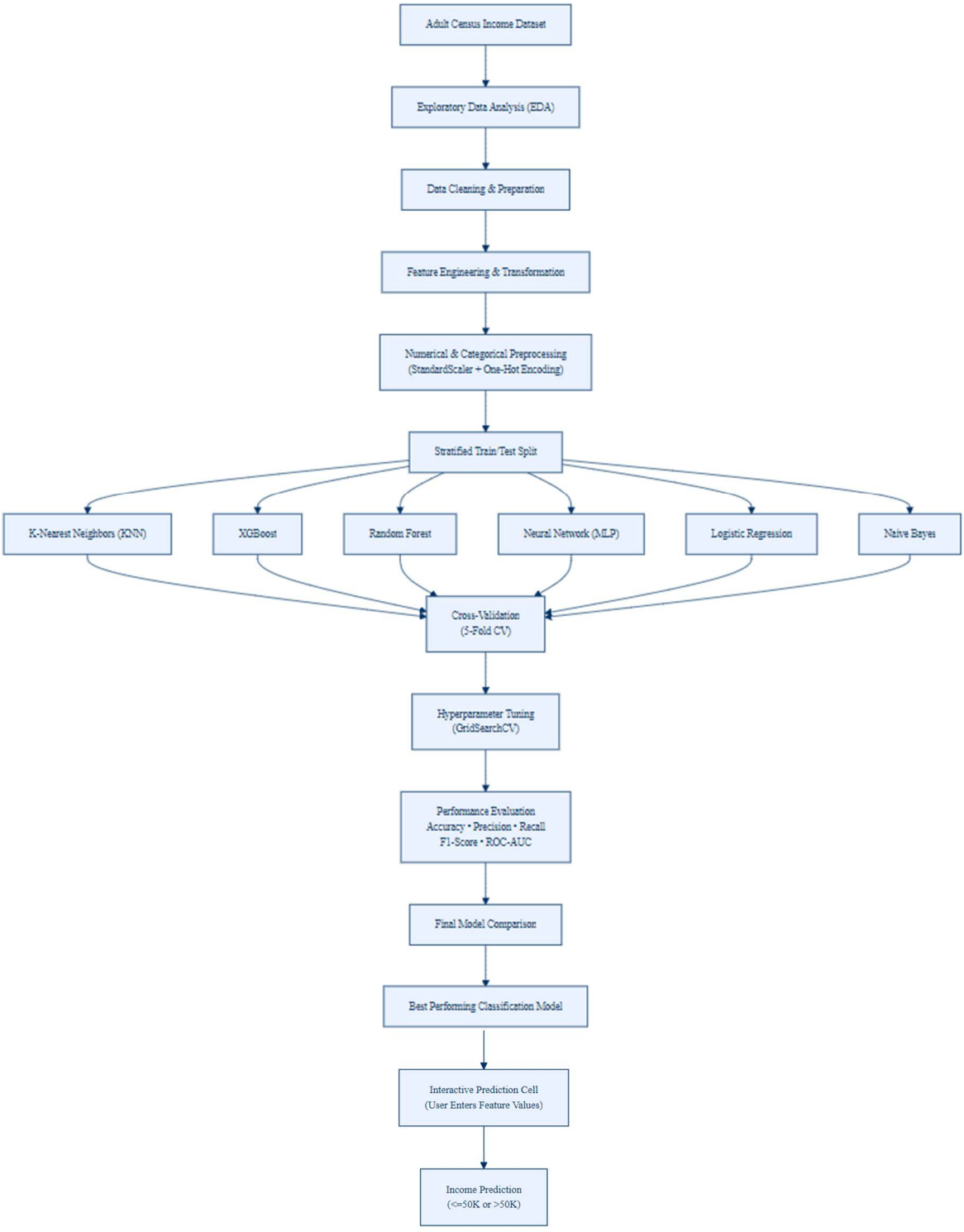


Figure 1. Machine Learning Workflow for the Adult Census Income Classification System

The workflow demonstrates that all classification algorithms were developed using the same preprocessing strategy and evaluation methodology, ensuring a fair and consistent comparison across all six machine learning models.

Conclusion

The Adult Census Income Classification System successfully demonstrates the complete development of a supervised machine learning application using the Python scientific computing ecosystem. Beginning with data exploration and preprocessing, the project progressed through feature transformation, model development, cross-validation, hyperparameter tuning, and comparative performance evaluation using six different classification algorithms.

The collaborative development approach allowed the project team to investigate multiple machine learning techniques while maintaining a common preprocessing pipeline and evaluation methodology. By comparing K-Nearest Neighbors, XGBoost, Random Forest, Neural Network, Logistic Regression, and Naive Bayes classifiers under identical experimental conditions, the team was able to evaluate the strengths and limitations of each algorithm objectively.

The implementation of an interactive prediction cell further extends the application beyond model development by allowing users to generate income predictions using custom input values. This feature demonstrates how the trained machine learning models can be applied to new observations and illustrates the practical deployment of the completed classification system.

Overall, the project provided valuable experience in data preprocessing, supervised machine learning, model evaluation, hyperparameter optimization, and collaborative software development while demonstrating the complete workflow required to design, implement, evaluate, and apply a real-world machine learning classification application.

References

Aurélien Géron. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems* (2nd ed.). O'Reilly Media.

Dua, D., & Graff, C. (2019). *UCI Machine Learning Repository*. University of California, Irvine, School of Information and Computer Sciences. https://archive.ics.uci.edu/

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

McKinney, W. (2022). *Python for Data Analysis* (3rd ed.). O'Reilly Media.